DIVIDE-AND-CONQUER

Divide and Conquer- Strategy

- For a function on *n* inputs, the inputs are partitioned into *k* distinct subsets, 1< k < n, yielding *k* subproblems.
- Solve the subproblems and then combine sub solutions into a solution of the entire problem.
- Repeatedly apply divide and conquer if the subproblems are relatively large.

Strategy (Cont'd)

- Continue until we get subproblems that are small enough to be solved without splitting further.
- We can write a control abstraction that mirrors the way an algorithm based on divide-and-conquer will look.
- Control abstraction defines the general flow of control while leaving specific operations undefined until applied to a specific problem.

```
1   Algorithm DAndC(P)
2   {
3       if Small(P) then return S(P);
4       else
5       {
6           divide P into smaller instances P_1, P_2, ..., P_k, k ≥ 1;
7           Apply DAndC to each of these subproblems;
8           return Combine(DAndC(P_1),DAndC(P_2),...,DAndC(P_k));
9       }
10  }
```

computing time of DAndC is described by the recurrence relation

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \cdots + T(n_k) \quad + \quad f(n) & \text{otherwise} \end{cases}$$

The complexity of many divide-and-conquer algorithms is given by recurrences of the form

$$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases} \tag{3.2}$$

BINARY SEARCH

```
1   Algorithm BinSrch(a, i, l, x)
2   // Given an array a[i : l] of elements in nondecreasing
3   // order, 1 ≤ i ≤ l, determine whether x is present, and
4   // if so, return j such that x = a[j]; else return 0.
5   {
6       if (l = i) then  // If Small(P)
7       {
8           if (x = a[i]) then return i;
9           else return 0;
10      }
11      else
12      { // Reduce P into a smaller subproblem.
13          mid := ⌊(i + l)/2⌋;
14          if (x = a[mid]) then return mid;
15          else if (x < a[mid]) then
16                  return BinSrch(a, i, mid − 1, x);
17              else return BinSrch(a, mid + 1, l, x);
18      }
19  }
```

Binary search

Find an element in a sorted array:

1. ***Divide:*** Check middle element.
2. ***Conquer:*** Recursively search 1 subarray.
3. ***Combine:*** Trivial.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Binary search

Find an element in a sorted array:

1. Divide: Check middle element.

2. Conquer: Recursively search 1 subarray.

3. Combine: Trivial.

Example: Find

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Binary search

Find an element in a sorted array:

1. Divide: Check middle element.

2. Conquer: Recursively search 1 subarray.

3. Combine: Trivial.

Example: Find

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Binary search

Find an element in a sorted array:

1. Divide: Check middle element.

2. Conquer: Recursively search 1 subarray.

3. Combine: Trivial.

Example: Find

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Binary search

Find an element in a sorted array:

1. Divide: Check middle element.

2. Conquer: Recursively search 1 subarray.

3. Combine: Trivial.

Example: Find

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Binary search

Find an element in a sorted array:

1. Divide: Check middle element.

2. Conquer: Recursively search 1 subarray.

3. Combine: Trivial.

Example: Find

9

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Binary search

Find an element in a sorted array:

1. Divide: Check middle element.

2. Conquer: Recursively search 1 subarray.

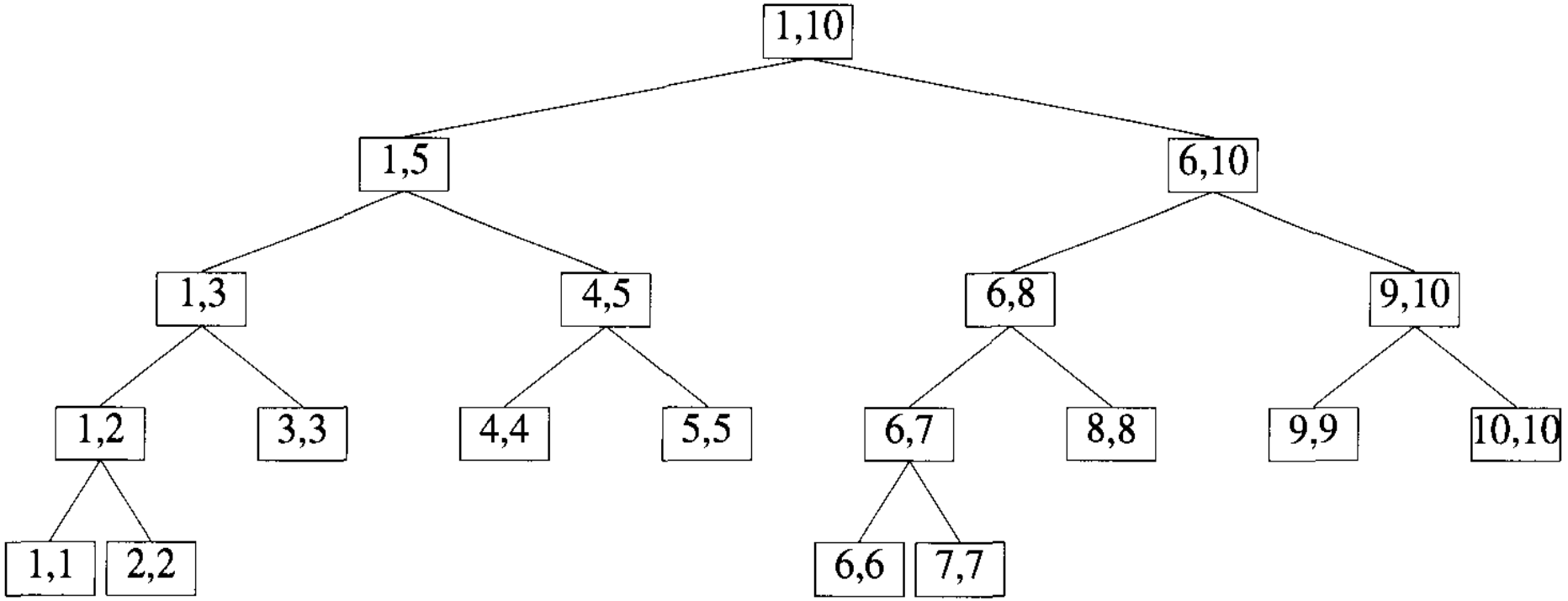
3. Combine: Trivial.

Example: Find

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

MERGE SORT

- Given a sequence of n elements a[1],...a[n].
- Split into two sets a[1]...a[n/2] and a[n/2+1],…a[n].
- Each set is individually sorted.
- The resulting sorted sequences are merged to produce a single sorted sequence of n elements.



Tree of calls of MergeSort$(1, 10)$

Consider the array of ten elements a[1:10] =

(310, 285,179, 652, 351, 423, 861, 254, 450, 520)

(310,285,179, 652, 351 | 423, 861,254, 450, 520)

(310, 285, 179 | 652,351 | 423,861,254,450,520)

(310, 285 | 179 | 652,351 | 423,861,254,450,520)

(310 | 285 | 179 | 652,351 | 423,861,254,450,520)

(285,310 | 179 | 652,351| 423, 861,254, 450,520)

(179,285,310 | 652,351 | 423,861,254, 450, 520)

(179,285,310 | 351,652 | 423,861,254, 450, 520)

(179,285,310,351,652 | 423, 861, 254, 450, 520)

(179,285,310,351,652 | 423, 861, 254 | 450,520)

(179,285,310,351,652 | 423, 861 | 254 | 450,520)

(179,285,310,351,652 | 423 | 861 | 254 | 450,520)

(179,285,310,351,652 | 423, 861 | 254 | 450,520)

(179,285,310,351,652 | 254, 423,861 | 450,520)

179,285,310,351,652 | 254, 423,861 | 450,520)

(179,285,310,351,652 | 254, 423, 450, 520, 861)

(179, 254, 285,310, 351, 423, 450,520, 652,861)

```
1   Algorithm MergeSort(low, high)
2   // a[low : high] is a global array to be sorted.
3   // Small(P) is true if there is only one element
4   // to sort. In this case the list is already sorted.
5   {
6       if (low < high) then  // If there are more than one element
7       {
8           // Divide P into subproblems.
9               // Find where to split the set.
10                  mid := ⌊(low + high)/2⌋;
11          // Solve the subproblems.
12              MergeSort(low, mid);
13              MergeSort(mid + 1, high);
14          // Combine the solutions.
15              Merge(low, mid, high);
16      }
17  }
```

```
1   Algorithm Merge(low, mid, high)
2   // a[low : high] is a global array containing two sorted
3   // subsets in a[low : mid] and in a[mid + 1 : high]. The goal
4   // is to merge these two sets into a single set residing
5   // in a[low : high]. b[ ] is an auxiliary global array.
6   {
7       h := low; i := low; j := mid + 1;
8       while ((h ≤ mid) and (j ≤ high)) do
9       {
10          if (a[h] ≤ a[j]) then
11          {
12              b[i] := a[h];h := h + 1;
13          }
14          else
15          {
16              b[i] := a[j]; j := j + 1;
17          }
18          i := i + 1;
19      }
20      if (h > mid) then
21          for k := j to high do
22          {
23              b[i] := a[k]; i := i + 1;
24          }
25      else
26          for k := h to mid do
27          {
28              b[i] := a[k]; i := i + 1;
29          }
30      for k := low to high do a[k] := b[k];
31  }
```

If the time for the merging operation is proportional to n, then the computing time for merge sort is described by the recurrence relation

$$T(n) = \begin{cases} a & n = 1, a \text{ a constant} \\ 2T(n/2) + cn & n > 1, c \text{ a constant} \end{cases}$$

QUICKSORT

- In merge sort, the array a[1:n] was divided at its midpoint into subarrays which were independently sorted and later merged.

- In quicksort, the division into two subarrays is made so that the sorted subarrays do not need to be merged later.

- The elements are rearranged by picking some pivot element of a, say t = a[s]
- Then reorder the other elements such that all elements appearing before t in a[1:n] are less than or equal to t and all elements appearing after t are greater than or equal to t.

This rearranging is referred to as **partitioning**.

(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	i	p
65	70	75	80	85	60	55	50	45	$+\infty$	2	9
65	45	75	80	85	60	55	50	70	$+\infty$	3	8
65	45	50	80	85	60	55	75	70	$+\infty$	4	7
65	45	50	55	85	60	80	75	70	$+\infty$	5	6
65	45	50	55	60	85	80	75	70	$+\infty$	6	5
60	45	50	55	65	85	80	75	70	$+\infty$		

```
1   Algorithm QuickSort(p, q)
2   // Sorts the elements a[p], ..., a[q] which reside in the global
3   // array a[1 : n] into ascending order; a[n + 1] is considered to
4   // be defined and must be ≥ all the elements in a[1 : n].
5   {
6       if (p < q) then  // If there are more than one element
7       {
8           // divide P into two subproblems.
9               j := Partition(a, p, q + 1);
10                  // j is the position of the partitioning element.
11          // Solve the subproblems.
12              QuickSort(p, j − 1);
13              QuickSort(j + 1, q);
14          // There is no need for combining solutions.
15      }
16  }
```

```
1   Algorithm Partition(a, m, p)
2   // Within a[m], a[m + 1], ..., a[p − 1] the elements are
3   // rearranged in such a manner that if initially t = a[m],
4   // then after completion a[q] = t for some q between m
5   // and p − 1, a[k] ≤ t for m ≤ k < q, and a[k] ≥ t
6   // for q < k < p. q is returned. Set a[p] = ∞.
7   {
8       v := a[m]; i := m; j := p;
9       repeat
10      {
11          repeat
12              i := i + 1;
13          until (a[i] ≥ v);

14          repeat
15              j := j − 1;
16          until (a[j] ≤ v);

17          if (i < j) then Interchange(a, i, j);

18      } until (i ≥ j);

19      a[m] := a[j]; a[j] := v; return j;
20  }
```

Worst-case partitioning

The worst-case behavior occurs when the partitioning routine produces one subproblem with n -1 elements and one with 0 elements.

The recurrence is

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) . \end{aligned}$$

$$T(0) = \Theta(1),$$

The average value $C_A(n)$ is computed under the assumption, that partitioning element *v* has an equal probability of being the ith-smallest element.

The recurrence is

$$C_A(n) = n + 1 + \frac{1}{n} \sum_{1 \leq k \leq n} [C_A(k-1)) + C_A(n-k)]$$

The number of element comparisons required by Partition on its first call is n + 1.

Note that $C_A(0) = C_A(1) = 0$.

Multiplying both sides of the recurrence by n, we obtain

$$nC_A(n) = n(n+1) + 2[C_A(0) + C_A(1) + \cdots + C_A(n-1)]$$

Replacing n by n-1, gives

$$(n-1)C_A(n-1) = n(n-1) + 2[C_A(0) + \cdots + C_A(n-2)]$$

Subtracting this from previous eqn,

$$nC_A(n) - (n-1)C_A(n-1) = 2n + 2C_A(n-1)$$

or

$$C_A(n)/(n+1) = C_A(n-1)/n + 2/(n+1)$$

Repeatedly using this equation to substitute for $C_A(n-1)$, $C_A(n-2)$,...

$$
\begin{aligned}
\frac{C_A(n)}{n+1} &= \frac{C_A(n-2)}{n-1} + \frac{2}{n} + \frac{2}{n+1} \\
&= \frac{C_A(n-3)}{n-2} + \frac{2}{n-1} + \frac{2}{n} + \frac{2}{n+1} \\
&\vdots \\
&= \frac{C_A(1)}{2} + 2\sum_{3 \le k \le n+1} \frac{1}{k} \\
&= 2\sum_{3 \le k \le n+1} \frac{1}{k}
\end{aligned}
$$

Since

$$\sum_{3 \le k \le n+1} \frac{1}{k} \le \int_2^{n+1} \frac{1}{x}\, dx = \log_e(n+1) - \log_e 2$$

$$C_A(n) \le 2(n+1)[\log_e(n+2) - \log_e 2] = O(n \log n)$$

Even though the worst-case time is $O(n^2)$, the average time is only O(n log n).

STRASSEN'S MATRIX MULTIPLICATION

The divide-and-conquer strategy suggests a way to compute the product of two n x n matrices. For simplicity we assume that n is a power of 2, that is, $n = 2^k$.

In case n is not a power of two, then enough rows and columns of zeros can be added so that the resulting dimensions are a power of two.

Imagine that A and B are each partitioned into four square submatrices, each submatrix having dimensions n/2 x n/2.

Then the product AB can be computed as

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

then

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

Since n is a power of 2, these matrix products can be recursively computed by the same algorithm we are using for the n x n case.

This algorithm will continue applying itself to smaller-sized submatrices until n becomes suitably small (n = 2) so that the product is computed directly.

To compute AB we need to perform eight multiplications of n/2 x n/2 matrices and four additions of n/2 x n/2 matrices.

Since two n/2 x n/2 matrices can be added in time cn^2 for some constant c, the overall computing time T(n) is given by the recurrence

$$T(n) = \begin{cases} b & n \leq 2 \\ 8T(n/2) + cn^2 & n > 2 \end{cases}$$

This solution is $T(n) = O(n^3)$.

Volker Strassen has discovered a way to compute C using only 7 multiplications and 18 additions or subtractions.

His method involves first computing the seven n/2 x n/2 matrices P, Q, R, S, T, U, and V as in:

$$
\begin{aligned}
P &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
Q &= (A_{21} + A_{22})B_{11} \\
R &= A_{11}(B_{12} - B_{22}) \\
S &= A_{22}(B_{21} - B_{11}) \\
T &= (A_{11} + A_{12})B_{22} \\
U &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
V &= (A_{12} - A_{22})(B_{21} + B_{22})
\end{aligned}
$$

Then C is computed using the formulas:

$$
\begin{aligned}
C_{11} &= P + S - T + V \\
C_{12} &= R + T \\
C_{21} &= Q + S \\
C_{22} &= P + R - Q + U
\end{aligned}
$$

The resulting recurrence relation for $T(n)$ is

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T(n/2) + an^2 & n > 2 \end{cases}$$

where a and b are constants. Working with this formula, we get

$$\begin{aligned} T(n) &= an^2[1 + 7/4 + (7/4)^2 + \cdots + (7/4)^{k-1}] + 7^k T(1) \\ &\leq cn^2(7/4)^{\log_2 n} + 7^{\log_2 n}, \ c \text{ a constant} \\ &= cn^{\log_2 4 + \log_2 7 - \log_2 4} + n^{\log_2 7} \\ &= O(n^{\log_2 7}) \approx O(n^{2.81}) \end{aligned}$$

Substitution method for solving recurrences

The substitution method for solving recurrences comprises two steps:

1. Guess the form of the solution.
2. Use mathematical induction to find the constants and show that the solution works.

This method is powerful, but we must be able to guess the form of the answer in order to apply it.

As an example, let us solve the recurrence

$T(n) = 2T(n/2) + n$

We guess that the solution is $T(n) = O(n \log n)$.

The substitution method requires us to prove that $T(n) <= cn \log n$ for an appropriate choice of the constant $c > 0$.

$T(n/2) <= c(n/2) \log (n/2)$.

Substituting into the recurrence yields:

$$T(n) <= 2c(n/2) \log (n/2) + n$$

$$= cn \log (n/2) + n$$

$$= cn \log n - cn \log 2 + n$$

$$= cn \log n - cn + n$$

$$<= cn \log n$$

Changing variables

Sometimes, a little algebraic manipulation can make an unknown recurrence similar to one you have seen before. As an example, consider the recurrence

$$T(n) = 2T\left(\lfloor \sqrt{n} \rfloor\right) + \lg n \ ,$$

which looks difficult. We can simplify this recurrence, though, with a change of variables. For convenience, we shall not worry about rounding off values, such as $\sqrt{n}$, to be integers. Renaming $m = \lg n$ yields

$$T(2^m) = 2T(2^{m/2}) + m \ .$$

We can now rename $S(m) = T(2^m)$ to produce the new recurrence

$$S(m) = 2S(m/2) + m \ ,$$

Indeed, this new recurrence has the same solution: $S(m) = O(m \lg m)$. Changing back from $S(m)$ to $T(n)$, we obtain

$$T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n) .$$

Solve:

- T(n) = T(n-1) + n
- T(n) = T(n/2) + 1

Recursion-tree method for solving recurrences

In a recursion tree, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.

We sum the costs within each level of the tree to obtain a set of per-level costs, and then we sum all the per-level costs to determine the total cost of all levels of the recursion.

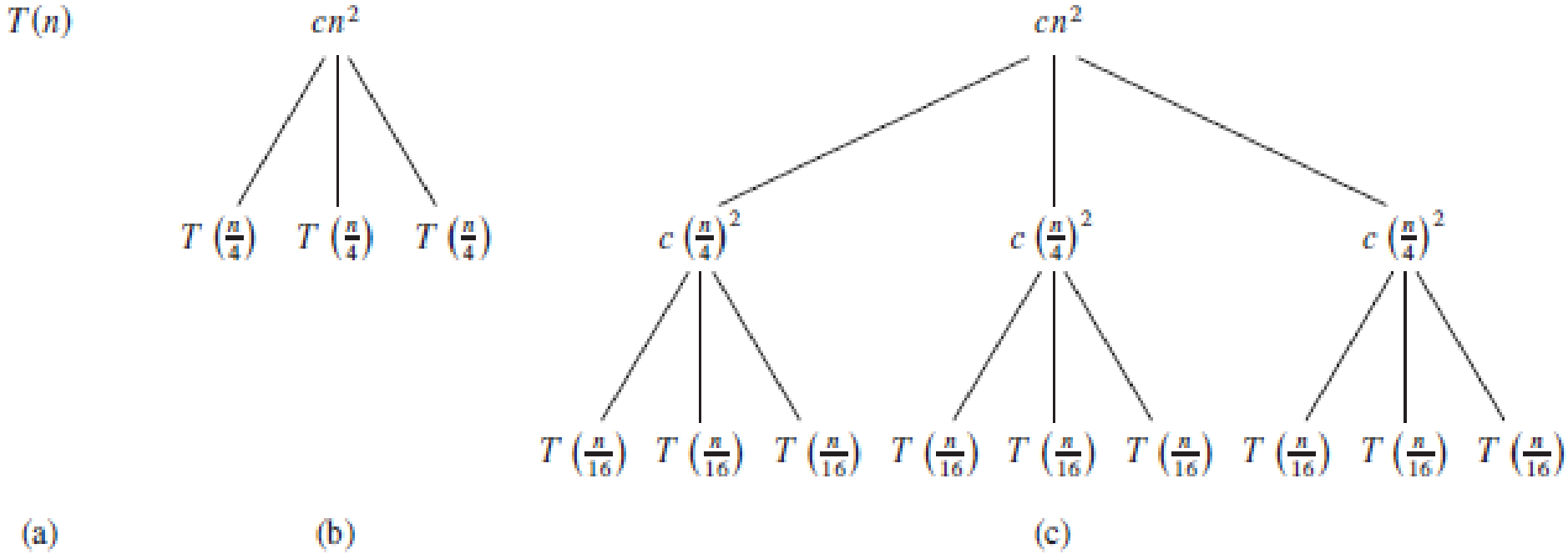
A recursion tree is best used to generate a good guess, which you can then verify by the substitution method.

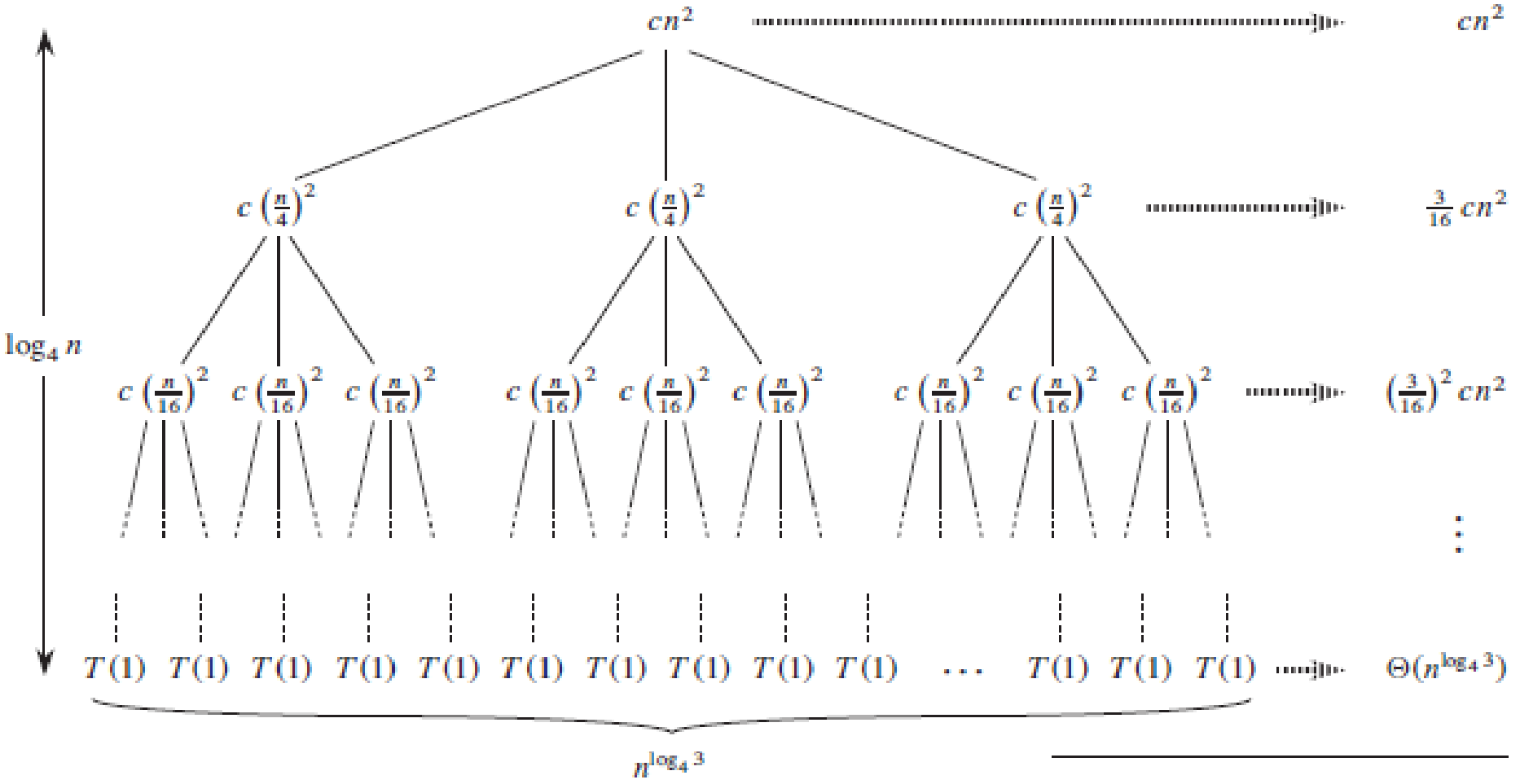
let us see how a recursion tree would provide a good guess for the recurrence

$T(n) = 3T(n/4) + \Theta(n^2)$

we create a recursion tree for the recurrence

$T(n) = 3T(n/4) + cn^2$, having constant, $c > 0$.


T(n)
cn²
T(n/4) T(n/4) T(n/4)
cn²
c(n/4)² c(n/4)² c(n/4)²
T(n/16) T(n/16) T(n/16) T(n/16) T(n/16) T(n/16) T(n/16) T(n/16) T(n/16)
(a)
(b)
(c)


cn^2
cn^2
$c\left(\frac{n}{4}\right)^2$
$c\left(\frac{n}{4}\right)^2$
$c\left(\frac{n}{4}\right)^2$
$\frac{3}{16}cn^2$
$\log_4 n$
$c\left(\frac{n}{16}\right)^2$
$\left(\frac{3}{16}\right)^2 cn^2$
$T(1)$
$\Theta(n^{\log_4 3})$
$n^{\log_4 3}$
Total: $O(n^2)$

(d)

$$
\begin{aligned}
T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\
T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{1}{1-(3/16)} cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\
&= O(n^2) \, .
\end{aligned}
$$

Now we can use the substitution method to verify that our guess was correct, that is, $T(n)= O(n^2)$.

We want to show that $T(n)<=dn^2$ for some constant $d > 0$.

$$
\begin{aligned}
T(n) &\leq 3T(\lfloor n/4 \rfloor) + cn^2 \\
&\leq 3d\,\lfloor n/4 \rfloor^2 + cn^2 \\
&\leq 3d(n/4)^2 + cn^2 \\
&= \frac{3}{16} dn^2 + cn^2 \\
&\leq dn^2 ,
\end{aligned}
$$

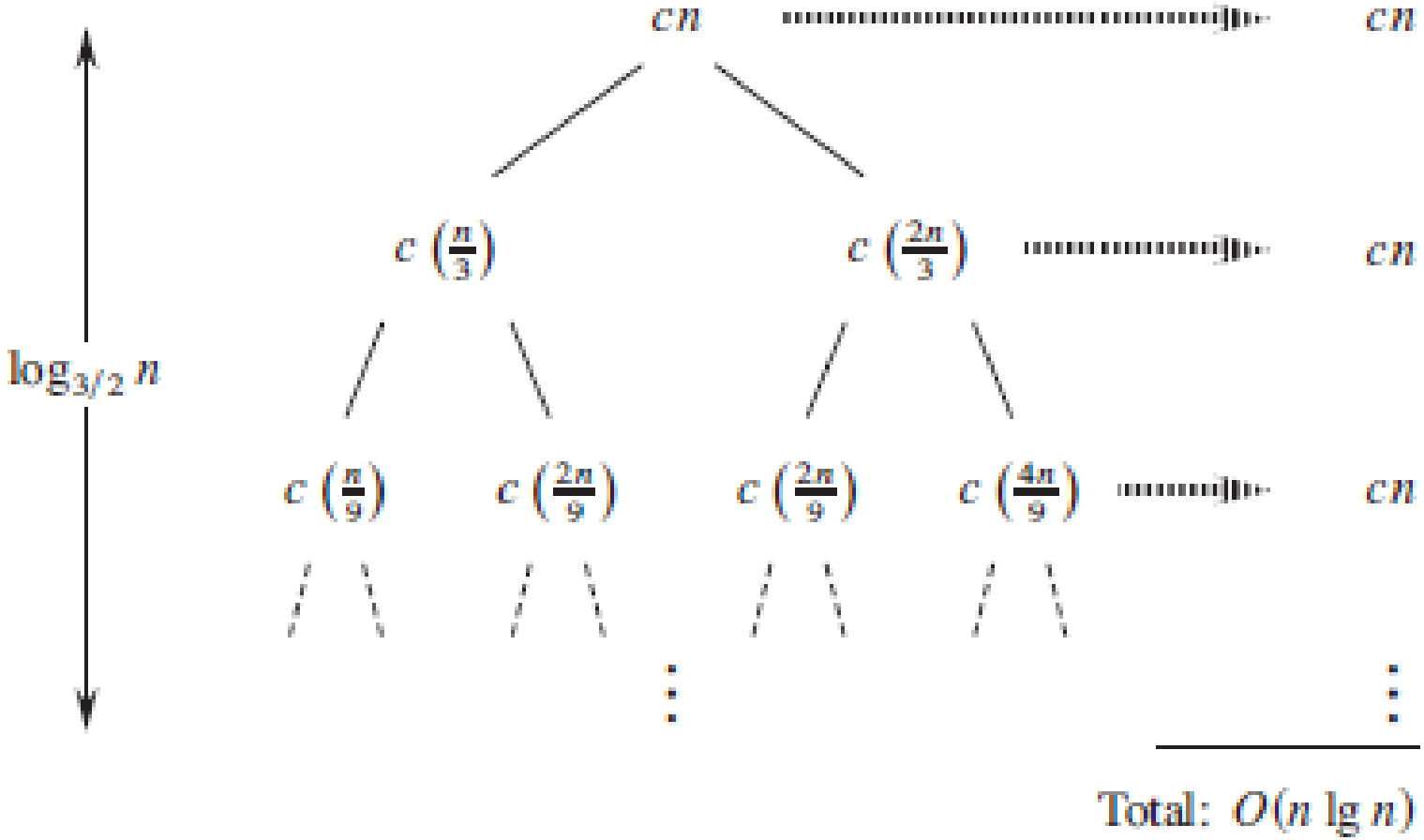
Another example:

$T(n) = T(n/3) + T(2n/3) + O(n)$

The longest simple path from the root to a leaf is $n \to (2/3)n \to (2/3)^2 n \to \cdots \to 1$. Since $(2/3)^k n = 1$ when $k = \log_{3/2} n$, the height of the tree is $\log_{3/2} n$.

The solution to the recurrence will be at most the number of levels times the cost of each level.

$$O(cn \log_{3/2} n) = O(n \lg n).$$


cn
cn
$c\left(\frac{n}{3}\right)$
$c\left(\frac{2n}{3}\right)$
cn
$\log_{3/2} n$
$c\left(\frac{n}{9}\right)$
$c\left(\frac{2n}{9}\right)$
$c\left(\frac{2n}{9}\right)$
$c\left(\frac{4n}{9}\right)$
cn
Total: $O(n \lg n)$

Now we can use the substitution method to verify that our guess was correct, T(n) = O(n log n).

We want to show that T(n) <= dn log n for some constant d > 0.

$$
\begin{aligned}
T(n) &\le T(n/3) + T(2n/3) + cn \\
&\le d(n/3)\lg(n/3) + d(2n/3)\lg(2n/3) + cn \\
&= (d(n/3)\lg n - d(n/3)\lg 3) \\
&\qquad + (d(2n/3)\lg n - d(2n/3)\lg(3/2)) + cn \\
&= dn\lg n - d((n/3)\lg 3 + (2n/3)\lg(3/2)) + cn \\
&= dn\lg n - d((n/3)\lg 3 + (2n/3)\lg 3 - (2n/3)\lg 2) + cn \\
&= dn\lg n - dn(\lg 3 - 2/3) + cn \\
&\le dn\lg n ,
\end{aligned}
$$